

Git Conflicto

Daniel Marcos Ferreira Castaño

1º DAM / ED

IES Torre de los Guzmanes

(2026-2027)

Índice

1. Crea repositorio local “practica-conflictos” con git init....3
2. Crea un archivo info.txt con contenido....3
3. Crea una rama, modifica info.txt y commit....4
4. Crea otra rama, modifica info.txt y commit....5
5. Fusiona la rama-a y rama-b, dándonos conflicto....6
6. Resuelve el conflicto manualmente....6
7. Marcar como conflicto resuelto....7
8. Mostrar y observar la gráfica del historial....7

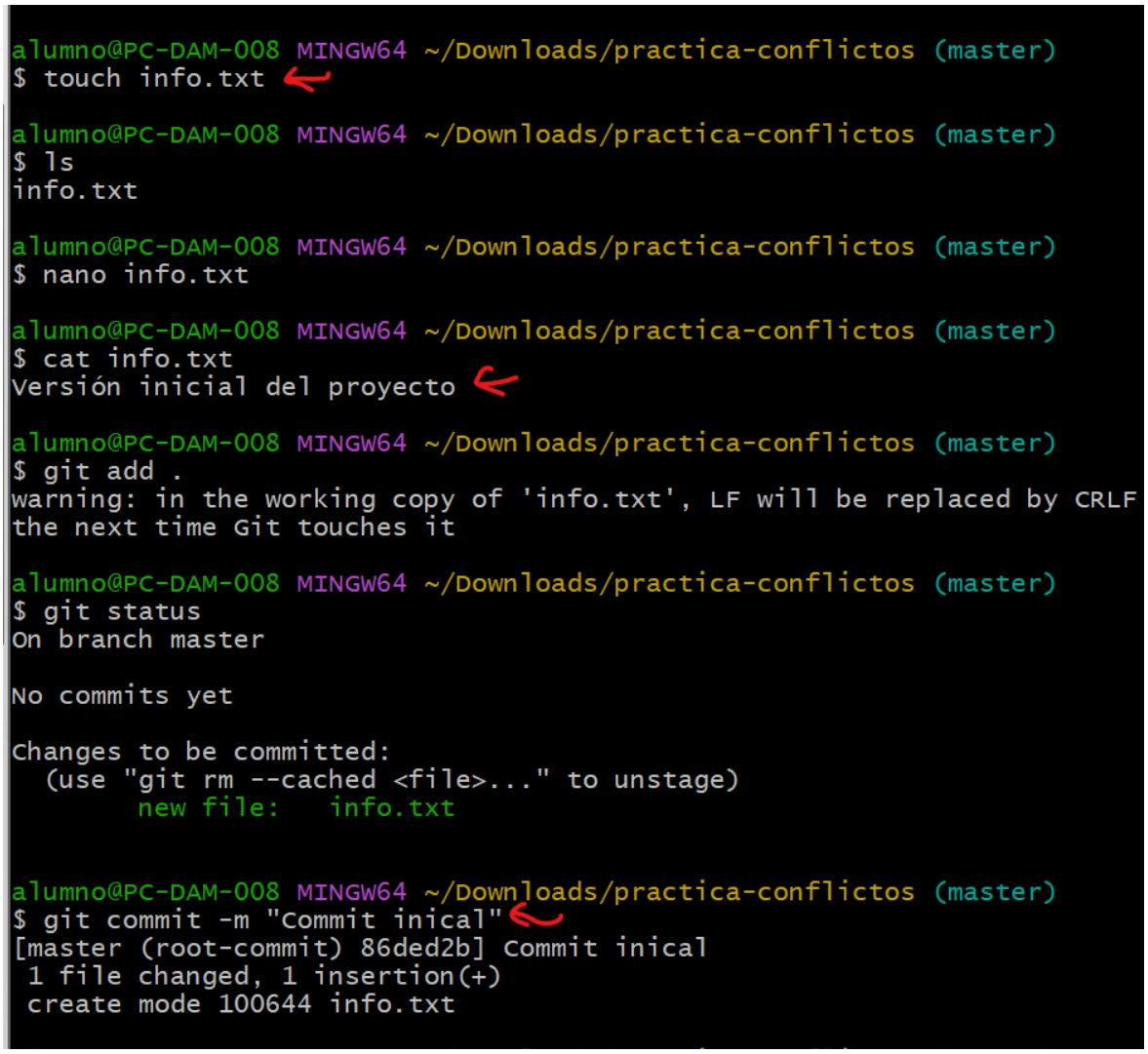
1. Crea repositorio local “practica-conflictos” con git init

Creamos un nuevo repositorio localmente ubicados donde nos interesa correctamente.



2. Crea un archivo info.txt con contenido

Creo el archivo pedido, le añado el contenido que indica el enunciado y hago commit de los cambios realizados.



3. Crea una rama, modifica info.txt y commit

Creo la rama “rama-a”, modifico la primera línea de info.txt y hago commit de nuevo.

```
alumno@PC-DAM-008 MINGW64 ~/Downloads/practica-conflictos (master)
$ git switch -c rama-a
Switched to a new branch 'rama-a'

alumno@PC-DAM-008 MINGW64 ~/Downloads/practica-conflictos (rama-a)
$ git branch -a
  master
* rama-a

alumno@PC-DAM-008 MINGW64 ~/Downloads/practica-conflictos (rama-a)
$ nano info.txt

alumno@PC-DAM-008 MINGW64 ~/Downloads/practica-conflictos (rama-a)
$ nano info.txt

alumno@PC-DAM-008 MINGW64 ~/Downloads/practica-conflictos (rama-a)
$ cat info.txt
Cambio realizado desde rama-a

alumno@PC-DAM-008 MINGW64 ~/Downloads/practica-conflictos (rama-a)
$ git add .
warning: in the working copy of 'info.txt', LF will be replaced by CRLF
the next time Git touches it

alumno@PC-DAM-008 MINGW64 ~/Downloads/practica-conflictos (rama-a)
$ git status
On branch rama-a
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   info.txt


alumno@PC-DAM-008 MINGW64 ~/Downloads/practica-conflictos (rama-a)
$ git commit -m "Commit desde rama"
[rama-a ec99bfd] Commit desde rama
 1 file changed, 1 insertion(+), 1 deletion(-)
```

4. Crea otra rama, modifica info.txt y commit

Hago los mismos pasos como el punto anterior cambiando el contenido de info.txt.

```
alumno@PC-DAM-008 MINGW64 ~/Downloads/practica-conflictos (rama-a)
$ git switch master
Switched to branch 'master'

alumno@PC-DAM-008 MINGW64 ~/Downloads/practica-conflictos (master)
$ git switch -c rama-b
Switched to a new branch 'rama-b'

alumno@PC-DAM-008 MINGW64 ~/Downloads/practica-conflictos (rama-b)
$ git branch -a
  master
  rama-a
* rama-b

alumno@PC-DAM-008 MINGW64 ~/Downloads/practica-conflictos (rama-b)
$ nano info.txt

alumno@PC-DAM-008 MINGW64 ~/Downloads/practica-conflictos (rama-b)
$ cat info.txt
Cambio realizado desde rama-b

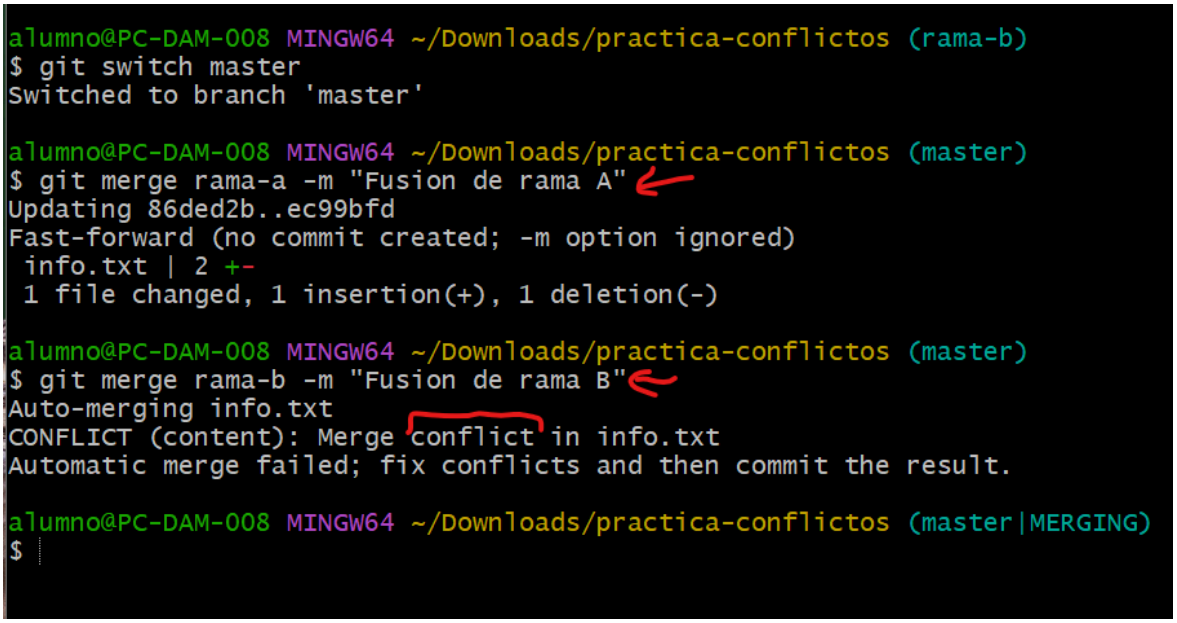
alumno@PC-DAM-008 MINGW64 ~/Downloads/practica-conflictos (rama-b)
$ git add .

alumno@PC-DAM-008 MINGW64 ~/Downloads/practica-conflictos (rama-b)
$ git status
On branch rama-b
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   info.txt


alumno@PC-DAM-008 MINGW64 ~/Downloads/practica-conflictos (rama-b)
$ git commit -m "Commit desde rama B"
[rama-b bfb6633] Commit desde rama B
 1 file changed, 1 insertion(+), 1 deletion(-)
```

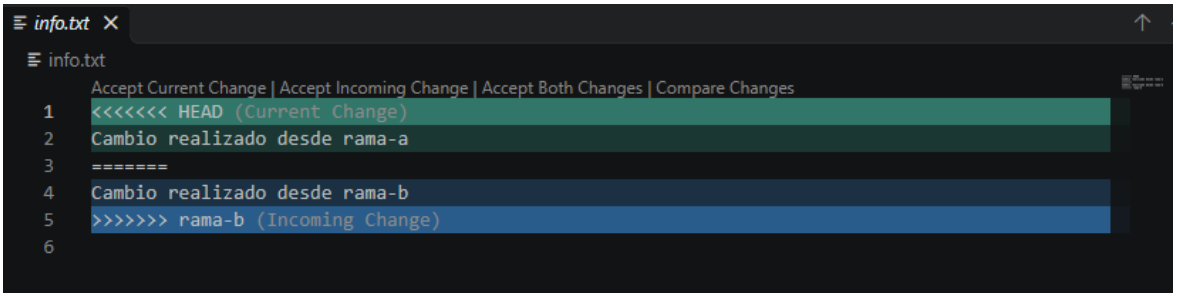
5. Fusiona la rama-a y rama-b, dándonos conflicto

Fusionamos ambas ramas al tronco y en la rama-b nos da conflicto como dicta la actividad correctamente.



6. Resuelve el conflicto manualmente

Ahora si abrimos el archivo que da conflicto, nos da la opción de elegir cual de los 2 contenidos nos interesa y guardamos. Así termina de fusionarse correctamente, quitándose "MERGING" que indica al lado de la rama en el bash.



7. Marcar como conflicto resuelto

Una vez elegido los cambios, lo añadimos e indicamos que está resuelto para terminar la fusión.

```
alumno@PC-DAM-008 MINGW64 ~/Downloads/practica-conflictos (master|MERGING)
$ git add .

alumno@PC-DAM-008 MINGW64 ~/Downloads/practica-conflictos (master|MERGING)
$ git commit -m "Resuelto conflicto fusion de ramas"
[master f52d352] Resuelto conflicto fusion de ramas

alumno@PC-DAM-008 MINGW64 ~/Downloads/practica-conflictos (master)
$
```

8. Mostrar y observar la gráfica del historial

Al ver el historial podemos observar los commits y fusión que se realizó para el conflicto. Aquí se puede apreciar mucho más claro cuando usas un "merge" que si se usara un "rebase" en la fusión. Merge y Rebase hacen lo mismo, fusionar, pero con una sutil diferencia:

- Merge fusiona e indica en el historial el último commit de dicha rama fusionada
- Rebase fusiona e indica en el historial TODOS los commits realizados a lo largo de dicha rama fusionada, dándonos más información pero quedando más "sucio" el historial, llegando ser abrumador y tal vez innecesario.

En este caso para el conflicto se optó por merge porque la actividad lo pedía, además que en realidad solo hacemos 1 commit osea que no tiene ninguna diferencia visual que pueda realizar… También se podría haber priorizado los cambios de la rama que el tronco, así evitando conflictos y directamente sustituyéndolo.

```
alumno@PC-DAM-008 MINGW64 ~/Downloads/practica-conflictos (master)
$ git log --oneline --graph --all
*   f52d352 (HEAD -> master) Resuelto conflicto fusion de ramas
|\
| * bfb6633 (rama-b) Commit desde rama B
* | ec99bfd (rama-a) Commit desde rama
|/
* 86ded2b Commit inical

alumno@PC-DAM-008 MINGW64 ~/Downloads/practica-conflictos (master)
$
```